

2019ab04045.pdf

by

Submission date: 11-Feb-2022 01:52PM (UTC+0530)

Submission ID: 1759943247

File name: 2019ab04045.pdf (2.84M)

Word count: 6138

Character count: 30782

A REPORT

ON

Time Series Forecasting Tool using MLOps

BY

Name of the : Rishav Saigal
Student

ID.No. : 2019AB04045

AT

Pilani (Rajasthan)

Capgemini Technology Services India Limited & Bengaluru

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

January, 2022

A REPORT

ON

Time Series Forecasting Tool using MLOps

BY

Name of the : Rishav Saigal
Student

ID.No. : 2019AB04045

Discipline : DSECLZG628T

Prepared in partial fulfillment of the
WILP Dissertation/Project/Project Work Course

AT

Capgemini Technology Services India Limited & Bengaluru

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

January, 2022

Acknowledgement

First, I would like to express my sincere gratitude to my mentor **Mr. Dhananjoy Mondal, Manager at Capgemini, Data Science & Analytics** for the continued support of my M.tech project and related research, for his immense patience, motivation and knowledge. His guidance helped me throughout the research and writing of this thesis. I couldn't imagine having a better mentor and environment like this.

I place on record, my profound sense of gratitude to **Mr. Raghvan P**, professor & department of Data Science & engineering, BITS Pilani (Work Integrated Learning Programme), to give me an opportunity to enhance my technical skills and valuable suggestions in this project.

19

I would like to thank my parents who helped me a lot in gathering different information, collecting data and guiding me from time to time in making this project, despite of their busy schedules, they gave me different ideas in making this project unique.

1
BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE
PILANI (RAJASTHAN)
WILP Division

Organization: Capgemini Technology Services India Limited... **Location:** Bengaluru

Duration 13 weeks **Date of Start** 25-Oct-2021

Date of Submission 26-Jan-2022

Title of the Project: Time Series Forecasting Tool using MLOps

ID No./Name of the student : 2019AB04045 / Rishav Saigal

**Name (s) and
Designation (s) of**

your Supervisor and Additional Examiner : Dhananjay Mondal, Manager at Capgemini, Data Science & Analytics

**Name of the
Faculty mentor:-** RAGHAVAN P

Key Words: Time Series, Forecasting, Docker, MLOps, Kubernetes, Distributed Training, KPI.

Project Areas: MLFlow and KubeFlow are the libraries built specifically for scaling the models and supporting the concept of MLOps but they are not specific to time series.

Abstract:

Time Series Forecasting has been a fundamental need in various practical domains such as retail, trade-finance, price prediction and many more. Many solutions have been built but they are still depending upon the old traditional models and lack of model monitoring KPIs. As we have seen in past few years, the exponential increase in data volume and frequent changes in data pattern which are affecting the longevity of model (viz. under-forecasting or over-forecasting). We require more mature solution that can scale up or down according to the volume of data and generate better models with advance MLOps metrics to provide stable predictions for long run.

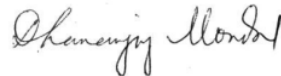
To solve this kind of problem we have come up with a solution introducing MLOps with Time Series in which we will be focusing specifically in two major drawbacks i.e the distributed training of models and long stability of the models using advance Drift KPIs. Let's say we have model 1 and model 2 in which both the models have different library requirements and computation time. In the above scenario, we can run them in a separate environment from our local system using docker or kubernetes pods. In our second scenario, we are considering the shifting of pattern in data over the course of time, our model needs to capture the latest trend; more importantly so to check the model stability we will be using MLOps monitoring KPIs.

Signature of Student



Date:- 22-Jan-2022

Signature of your Supervisor



Date:- 22-Jan-2022

Table of Contents

1. Introduction	6
2. Overall Task Report	7
3. Architecture	8
3.1. Database & Table Design	8
3.2. Project Design	12
3.3. API Design	13
3.4. Data Pre-processing Pipeline	15
3.5. Model Training Pipeline	16
3.6. Web Front-end (using React) & Performance Visualization (using Stream-lit)	17
4. Algorithms	18
4.1. Outlier Correction Techniques	18
4.1.1. Local Outlier Factor	18
4.1.2. Isolation Forest Outlier Correction	18
4.1.3. Z-Score Outlier Correction	18
4.2. Imputation Techniques	18
4.2.1. Mean	18
4.2.2. Median	18
4.2.3. Linear	18
4.2.4. Nearest	19
4.3. Time Series Models	19
4.3.1. ARIMA	19
4.3.2. Exponential Smoothing	19
4.3.3. Prophet	19
4.3.4. Naive	19
4.3.5. Theta	19
4.3.6. Poly-Trend	19
4.3.7. Ensemble	19
4.3.8. Auto Ensemble	20
4.3.9. Time Series Splitter	20
4.3.9.1. Expanding Window	20
4.3.9.2. Sliding Window	20
4.4. Scoring Techniques	20
4.4.1. Concept Drift Metrics	20
4.4.1.1. Kolmogorov Smirnov (K-S)	21
4.4.1.2. Population stability index (PSI)	21
4.4.2. Performance Metrics	21
4.4.2.1. Mean Absolute Percentage Error (MAPE)	21
4.4.2.2. Akaike Information Criterion (AIC)	22
4.4.2.3. Bayesian Information Criterion (BIC)	22
4.4.2.4. Bias	22
4.4.2.5. Root Mean Squared Error (RMSE)	22
4.4.2.6. Root Mean Percentage Squared Error (RMSPE)	23
4.4.3. Best Model Selection Scoring Method	23
5. Deployment & Scalability	24
6. Demo Screens & Video	26
7. Conclusions and Recommendations	32
8. Reference	33
9. Duly Completed Checklist	34

1 - Introduction

11

Analyzing time-oriented data and forecasting are among the most important problems that analysts face across many fields, ranging from finance and economics to production operations and the natural sciences. As a result, there is a widespread need for large groups of people in a variety of fields to understand the basic concepts of time series analysis and forecasting.

21

The applications of time series models are many and wide-ranging, from sales forecasting to weather forecasting. In decisions that involve a factor of uncertainty about the future, time series models have been found to be among the most effective methods of forecasting.

This solution specifically focuses on two areas specifically the scalability of the time series forecasting in parallel and support distributed computing , and second one focuses on concept drift i.e. changes in data pattern over time in unforeseen ways. This causes problems and reduces the prediction accuracy as time passes.

We have now understood what are the key importance of time series forecasting in the field of different sectors and even got the idea about the problem statement. How should we achieve it ? Before asking this questions let's understand. What do you mean by Parallel Processing, Distributed Computing and MLOps.

A Parallel Processing system is able to perform concurrent data processing to achieve faster execution time. Increasing the computational speed of a computer system that cooperate to solve large computational problems fast by dividing such problems into parallel tasks, exploiting Thread-Level Parallelism (TLP).

12

Distributed computing (or distributed processing) is the technique of linking together multiple computer servers over a network into a cluster, to share data and to coordinate processing power. Such a cluster is referred to as a "distributed system." Distributed computing offers advantages in scalability (through a "scale-out architecture"), performance (via parallelism), resilience (via redundancy), and cost-effectiveness (through the use of low-cost, commodity hardware).

14

MLOps is a set of practices for collaboration and communication between data scientists and operations professionals. Applying these practices increases the quality, simplifies the management process, and automates the deployment of Machine Learning and Deep Learning models in large-scale production environments. It's easier to align models with business needs, as well as regulatory requirements.

2 - Overall Task Report

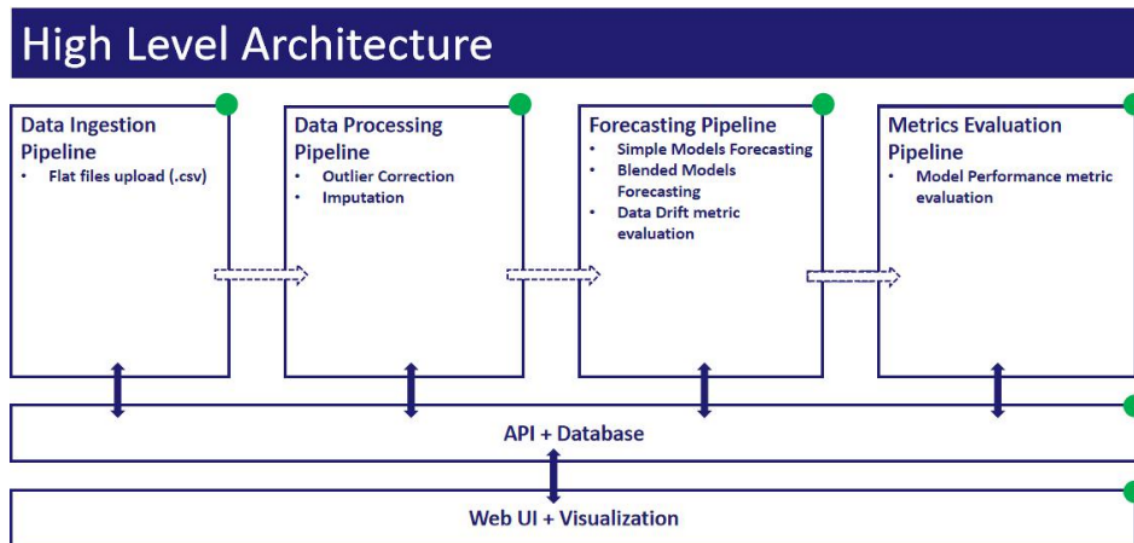
#	Task	Expected date of completion	Names of Deliverable	Status
1	Detailed Functional Requirement, Workflow and Data-flow	30-Oct-2021	Workflow Design	Completed
2	Design and Build Tables	1-Nov-2021	Database Architecture	Completed
3	Outlier Correction module, Time-Series Split module, Imputation module	4-Nov-2021	Data Processing Module	Completed
4	5 Forecasting models to be build + 2 Blended Models Architecture	12-Nov-2021	Forecasting Modules	Completed
5	Module to be build AIC, MAPE, BIAS, MSE, R2	14-Nov-2021	Metrics Module	Completed
6	Module to be built for PSI and DI	18-Nov-2021	Model Performance Stability Metrics	Completed
7	data processing pipeline, forecasting pipeline, metrics pipeline	29-Nov-2021	Pipeline Creation	Completed
8	Testing end to end of the modules build	4-Dec-2021	Testing Phase	Completed
9	Creation of mid semester report, test results and overall completion status	9-Dec-2021	Mid Semester Report	Completed
10	Data Ingestion, Configuration Page (Data Processing, Forecasting), Process Monitor Page, Model Performance Page	6-Jan-2022	UI Design	Completed
11	Building REST APIs and Integration of it with UI	10-Jan-2022	Python Back-end	Completed
12	Building docker modules for each modules scaling	14-Jan-2022	Containerizing the modules	Completed
13	Testing end to end of the modules	19-Jan-2022	Testing Phase	Completed
14	Creation of final semester report, test results and overall completion status	24-Jan-2022	Final Semester Report	Completed

Modifications :

1. Added BIC , RMSE and RMSPE metrics and removed R2 and MSE from metric calculation.
2. Instead of DI we are using KS (Kolmogorov–Smirnov test) to track the data drift.
3. Added 1 model to Simple models previously planned 5 added one more so total 6 models and 2 blended models.

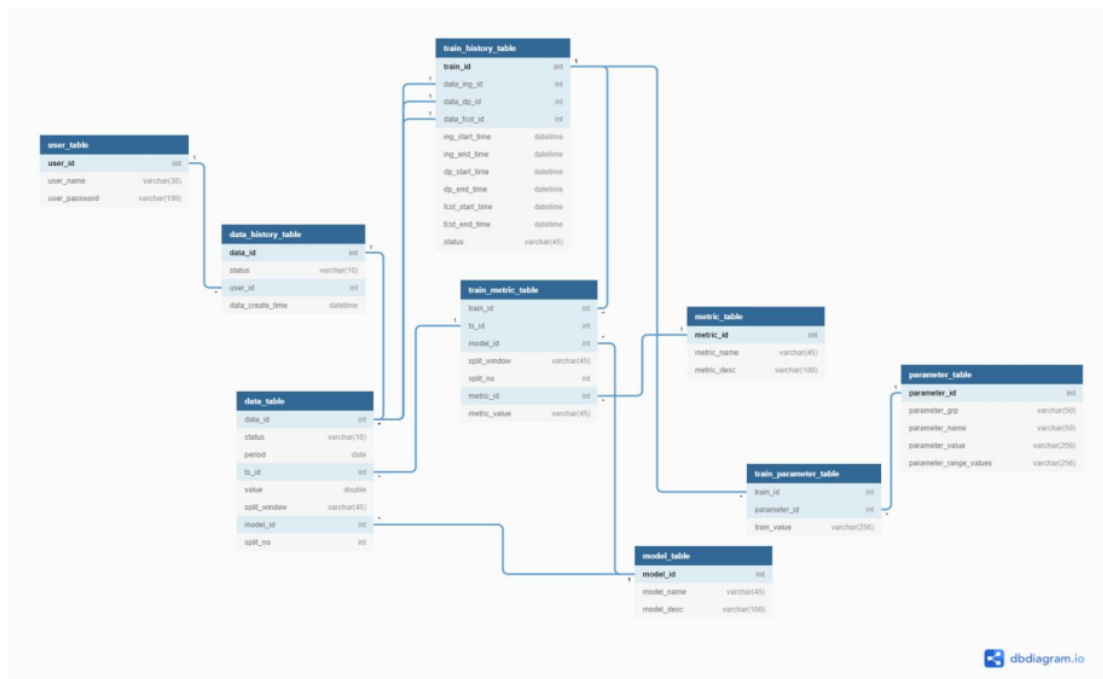
3 - Architecture

Below is shown the high level architecture.



First we will be understanding the database & table architecture and the technology used behind. Here we have used MySQL database.

3.1 - Database & Table Design



3.1.1 - User_table:

user_id
user_name
user_password

	user_id	user_name	user_password
▶	1	rishavsaigal	a1e9688716e816f133e681384ddf1536
*	NULL	NULL	NULL

This table is used for storing the user information, and the data and all further training information are stored at user level.

3.1.2 - Data_history_table:

data_id
status
user_id
data_create_time

	data_id	status	user_id	data_create_time
▶	1	ING	1	2022-01-16 11:47:23
	2	DP	1	2022-01-16 11:47:43
	3	FCST	1	2022-01-16 11:48:09
	4	ING	1	2022-01-16 11:49:44
	5	DP	1	2022-01-16 11:49:54
	6	FCST	1	2022-01-16 11:50:29
	7	DP	1	2022-01-18 11:41:40
	8	FCST	1	2022-01-18 11:42:11
	9	FCST	1	2022-01-18 11:44:31
	10	FCST	1	2022-01-18 11:48:45

This table is used for storing the data information , when the data id was created and what type if data it is like ING (ingestion), DP (data pre-processing) and FCST (forecasting) along with user id. Multiple data ids for a particular are done to keep in mind of reusing the older runs for further process with different parameters, which we will see in further when we go with demo.

3.1.3 - Data_table:

data_id
status
period
ts_id
value
split_window
model_id
split_no

This table holds all the time series information like data_id, status, ts_id (time series id you can pass multiple time series), period (date time) and value (actual historical sale, corrected sales or forecast depending on which stage it is). Split Window gives which type of window we are using like expanding or sliding window for training. Split No it shows no of splits of that type. Model ID stores the information of what model was used.

	data_id	status	period	ts_id	value	split_window	model_id	split_no
▶	3	FCST	1984-05-31	1	263	expanding	1	0
	3	FCST	1984-06-30	1	222	expanding	1	0
	3	FCST	1984-07-31	1	223	expanding	1	0
	3	FCST	1984-08-31	1	256	expanding	1	0
	3	FCST	1984-09-30	1	315	expanding	1	0
	3	FCST	1984-10-31	1	366	expanding	1	1
	3	FCST	1984-11-30	1	396	expanding	1	1
	3	FCST	1984-12-31	1	454	expanding	1	1
	3	FCST	1985-01-31	1	441	expanding	1	1
	3	FCST	1985-02-28	1	414	expanding	1	1
	3	FCST	1985-03-31	1	393	expanding	1	2
	3	FCST	1985-04-30	1	330	expanding	1	2
	3	FCST	1985-05-31	1	300	expanding	1	2

3.1.4 - Parameter_table:

parameter_id
parameter_grp
parameter_name
parameter_value
parameter_range_values

This table holds all the default parameters, like stage information, outlier correction, imputation methods , model lists , test size, spit windows , no of splits of the time series for cross validation and others.

parameter_id	parameter_grp	parameter_name	parameter_value	parameter_range_values
1	INGESTION_STAGE	ing_start	ING_S	NULL
2	INGESTION_STAGE	ing_processing	ING_P1	NULL
3	INGESTION_STAGE	ing_end	ING_E	NULL
4	INGESTION_STAGE	ing_failed	ING_F	NULL
5	DATA_PROCESSING_STAGE	data_proc_start	DP_S	NULL
6	DATA_PROCESSING_STAGE	data_proc_processing	DP_P1	NULL
7	DATA_PROCESSING_STAGE	data_proc_end	DP_E	NULL
8	DATA_PROCESSING_STAGE	data_proc_failed	DP_F	NULL
9	FORECAST_STAGE	fcst_start	FCST_S	NULL
10	FORECAST_STAGE	fcst_processing	FCST_P1	NULL
11	FORECAST_STAGE	fcst_end	FCST_E	NULL
12	FORECAST_STAGE	fcst_failed	FCST_F	NULL
13	WINDOW_SPLITTER	sliding_window_splitter	SLIDE	NULL
14	WINDOW_SPLITTER	expanding_window_splitter	EXPAND	NULL
15	INGESTION_STAGE	ing_flag	ING	NULL
16	DATA_PROCESSING_STAGE	data_proc_flag	DP	NULL
17	FORECAST_STAGE	fcst_flag	FCST	NULL
18	IMPUTER	impute_choice	linear	linear,mean,median,nea...
19	IMPUTER	impute_if_zero	False	True,False
20	OUTLIER	outlier_choice	if	if,lof,zscore

3.1.5 - Model_table:

model_id
model_name
model_desc

This table holds the information of list of models that are currently supported by the solution.

model_id	model_name	model_desc
1	arima	simple
2	ets	simple
3	naive	simple
4	poly_trend	simple
5	prophet	simple
6	theta	simple
7	auto_ensemble	ensemble
8	ensemble	ensemble

3.1.6 - Metric_table:

metric_id
metric_name
metric_desc

This table it holds the information of all the metrics that are there in solution.

metric_id	metric_name	metric_desc
1	rmse	performance
2	rmspe	performance
3	aic	performance
4	bic	performance
5	bias	performance
6	mape	performance
7	psi	data_drift
8	ks	data_drift

3.1.7 - Train_history_table:

train_id
data_ing_id
data_dp_id
data_fcst_id
ing_start_time
dp_start_time
fcst_start_time
ing_end_time
dp_end_time
fcst_end_time
status

This table holds the all the information of three major process in solution of a particular train i.e. ingestion, data pre-processing and forecasting and the respective data ids, start and end time of the finish and status shows what is the final state of that particular train.

	train_id	data_ing_id	data_dp_id	data_fcst_id	ing_start_time	ing_end_time	dp_start_time	dp_end_time	fcst_start_time	fcst_end_time	status
▶	1	1	2	3	2022-01-16 11:47:23	2022-01-16 11:47:23	2022-01-16 11:47:43	2022-01-16 11:47:43	2022-01-16 11:48:09	2022-01-16 11:48:49	FCST_E
	2	4	5	6	2022-01-16 11:49:44	2022-01-16 11:49:44	2022-01-16 11:49:54	2022-01-16 11:49:54	2022-01-16 11:50:29	2022-01-16 11:51:08	FCST_E
	3	1	7	8	2022-01-18 11:41:40	2022-01-18 11:41:40	2022-01-18 11:41:40	2022-01-18 11:41:40	2022-01-18 11:42:11	2022-01-18 11:42:19	FCST_E
	4	1	7	9	2022-01-18 11:44:31	2022-01-18 11:44:31	2022-01-18 11:44:31	2022-01-18 11:44:31	2022-01-18 11:44:31	2022-01-18 11:45:29	FCST_E
	5	1	7	10	2022-01-18 11:48:44	2022-01-18 11:48:44	2022-01-18 11:48:44	2022-01-18 11:48:44	2022-01-18 11:48:45	2022-01-18 11:49:12	FCST_E

3.1.8 - Train_parameter_table:

train_id
parameter_id
parameter_name
parameter_value

This table holds the information of all the parameters specific for a train id.

	train_id	parameter_id	train_value
▶	1	18	nearest
	1	19	0
	1	20	if
	1	21	auto
	1	22	3
	1	23	0.4
	1	24	0.2
	1	25	multi
	1	26	1,2,3,4,5,6
	1	27	1
	1	28	1
	1	29	both

3.1.9 - Train_metric_table:

train_id
ts_id
model_id
split_window
split_no
metric_id
metric_value

This table holds the information of performance and concept drift metrics.

	train_id	ts_id	model_id	split_window	split_no	metric_id	metric_value
▶	1	1	2	expanding	0	7	6.6871034586
	1	1	2	expanding	0	8	0.0035309746
	1	1	3	expanding	0	7	6.6871034586
	1	1	3	expanding	0	8	0.0035309746
	1	1	4	expanding	0	7	6.6871034586
	1	1	3	expanding	1	7	6.1220084343
	1	1	2	expanding	1	7	6.1220084343
	1	1	2	expanding	1	8	0.0282419906
	1	1	3	expanding	1	8	0.0282419906
	1	1	4	expanding	0	8	0.0035309746
	1	1	4	expanding	1	7	6.1220084343
	1	1	3	expanding	2	7	6.1821360464
	1	1	4	expanding	1	8	0.0282419906
	1	1	3	expanding	2	8	0.5906226383

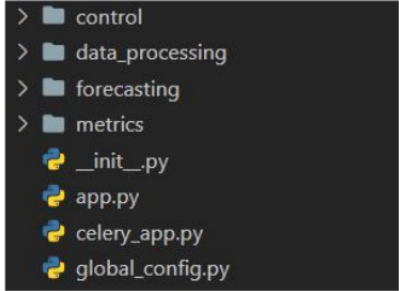
3.2 - Project Design

In this section we are going to talk about, how the project structure looks like. This would be a high level understanding what these modules are build for

- ✓ backend
- ✓ frontend
- ✓ mysql
- ✓ python_image
- ✓ python_visual_image
- ✓ visualization

Here in the **backend** folder it consists of the REST APIs, ingestion pipeline, data processing pipeline, forecasting pipeline and metrics calculations models.

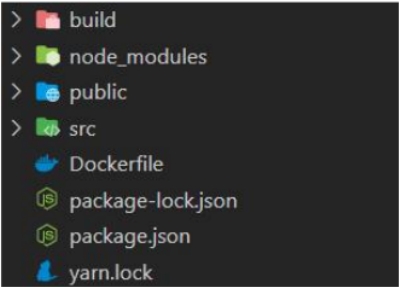
- app.py holds all the API requests functions.
- celery_app.py is used for scaling the queues in celery workers
- global_config.py holds the configuration info's database ports and hosts.
- control consists of all the functions that are required for API requests



```
> control
> data_processing
> forecasting
> metrics
  __init__.py
  app.py
  celery_app.py
  global_config.py
```

Under **frontend** folder we have all the modules for the Web UI Design.

- build is for the production version.
- public and src are the part of the development.
- node_modules are for the purpose of storing the support libraries which are required for development.
- Dockerfile is used for deployment in a remote server.
- package.json is used for the information of all the supported libraries.



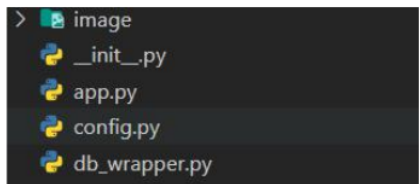
```
> build
> node_modules
> public
> src
  Dockerfile
  package-lock.json
  package.json
  yarn.lock
```

In **mysql** we have stored the create scripts for both database and tables further there are some default values that are need to be pre-populated before the application starts.



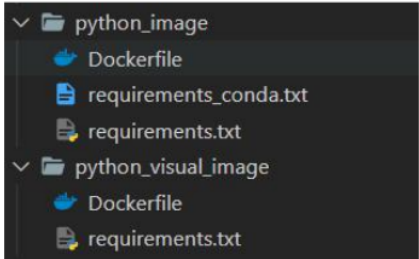
```
> scripts
  Dockerfile
```

Under **visualization** we have stream-lit application written in python, the sole purpose for this application is for inferring the model performance, past historical data and best model scoring techniques.



```
> image
  __init__.py
  app.py
  config.py
  db_wrapper.py
```

Under **python_image** and **python_visual_image** both are used to create docker images and consists of requirements.txt. This is used for building images that support all the above sub modules to run smoothly on the docker or kubernetes environment.



```
✓ python_image
  Dockerfile
  requirements_conda.txt
  requirements.txt
✓ python_visual_image
  Dockerfile
  requirements.txt
```

3.3 - API Design

Here we are going to talk about the APIs that were built to support the solution.

App.py - This file is the starting point of the application, here we have written route functions

- a) /test - this is used for testing if the app is running successfully.
- b) /signup - in this route we are allowing the users to insert a new user, if the user exists it will throw an error to the user as a response else it will insert and show a successful message.
 - i. Inputs required username and password
 - ii. Output error if same username it will fail else it will return a successful message.
- c) /login - in this route we are using to verify the user.
 - i. Inputs required username and password
 - ii. Output error if username doesn't exist else it will return user id on success and this will be used for further in all the APIs
- d) /upload-ingestion-data - in this route we are uploading the time series data that will ingest the data to data_table and store it with a specific train id and data id. Here we require the data in specific format i.e. file format should be csv and it should have ts_id, period and value. And from data science perspective minimum 2 years of data and recommended is 3 years of data.
- e) /get-default-data-processing-parameters - in this route we are fetching all the default parameters of data pre-processing techniques.
- f) /get-dp-train-ids - in this route we are fetching train type and user id and in return we will be returning the train ids, train types are either create or existing, in create we have new train ids that were not processed early and in existing it will use older train id where data is already processed and that copy of data will be further used for data pre-processing.
- g) /trigger-data-processing - in this route we are fetching train id, parameters chosen for data pre-processing and user id. And triggering the data pre-processing pipeline, which we will further discuss in the data pre-processing section.
- h) /get-default-forecasting-parameters - in this router we returning the list of forecasting parameters.
- i) /get-fcst-train-ids - this route is same as /get-dp-train-ids but only difference is, it will fetch only the data processed stage train ids and not other stages.
- j) /trigger-forecasting - it is same as /trigger-data-processing where we will trigger forecasting pipeline. We will study in detail in the forecasting pipeline and its process.
- k) /trigger-metrics-calculation - this is used for calculating all the performance metrics
- l) /status - this is used for fetching all the status of train ids listed under the user id.

Celery_app.py - this file is used for configuring the celery pipeline, here we will be running data pre-processing modules and forecasting modules.

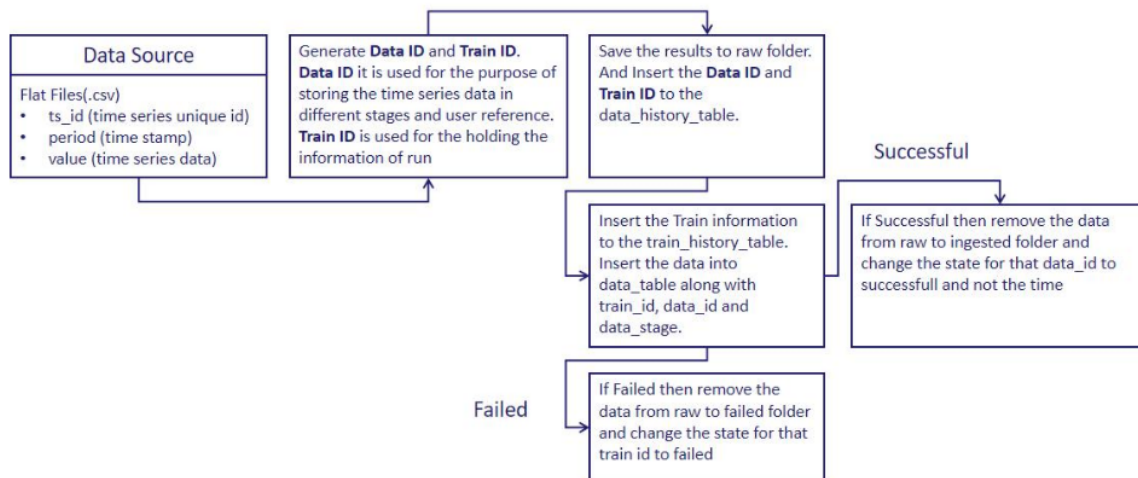
Global_config.py - here in this file we will be specifying the host and ports of mysql and same for redis, here redis is been used as a caching services in celery.

Control - Below you can see the folder structure

```
> failed
> ingested
> raw_data
  __init__.py
  control_config.py
  control_helper.py
  control_modal.py
  db_modal.py
```

After the routing APIs we are now focusing on ingestion, below we have added the ingestion flow structure and its components used in the process.

Ingestion Work-Flow



Control_config.py - in this file we provide all the solutions intermediate stages and parameter information.

Control_helper.py - in this file we have database connectivity and all the helper functions required in both API routes and control_modal.py

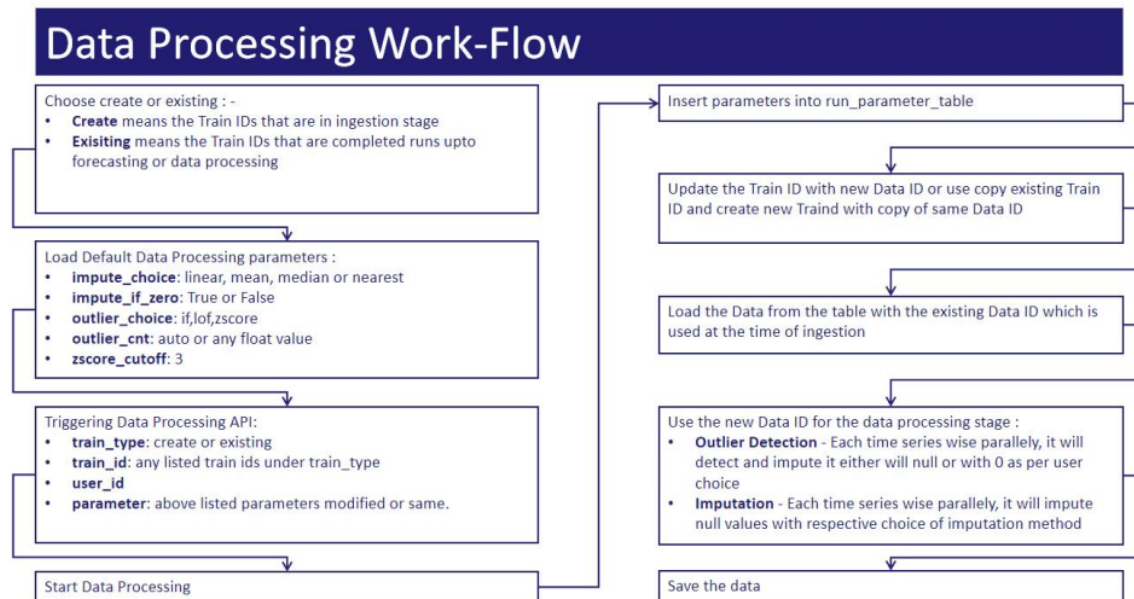
Control_modal.py - in this file we have setup the functions required by APIs.

- i. generate_ingest_id - here we ingest train id and data id
- ii. ingest_parameters - here we ingest parameters both used for data pre-processing and forecasting parameters ingestion
- iii. ingest_data - in this we follow the above workflow to ingest the data into the table
- iv. data_preprocessing_pipeline - here we trigger the data processing pipeline which further triggers the celery app
- v. forecasting_pipeline - here we trigger the forecasting pipeline which further triggers the celery app.

ingested, raw_data and failed are the folders required during ingestion, first we load the data into raw folder which further referred in ingest_data function to load the data into table if it is successfully loaded the data is then copied to ingested along with data id and deleted from raw folder if it fails it will delete from raw folder to failed along with data id.

3.4 - Data Pre-processing Pipeline

Lets have a look on the data pre-processing workflow.



```

data_processing
├── __pycache__
├── imputers
│   ├── __pycache__
│   ├── __init__.py
│   ├── linear_imputer.py
│   ├── mean_imputer.py
│   ├── median_imputer.py
│   └── nearest_imputer.py
├── outliers
│   ├── __pycache__
│   ├── __init__.py
│   ├── isolation_forest_detector.py
│   ├── local_outlier_factor_detector.py
│   ├── zscore_detector.py
│   ├── __init__.py
│   ├── data_processing_config.py
│   ├── data_processing_helper.py
│   └── data_processing_pipeline.py

```

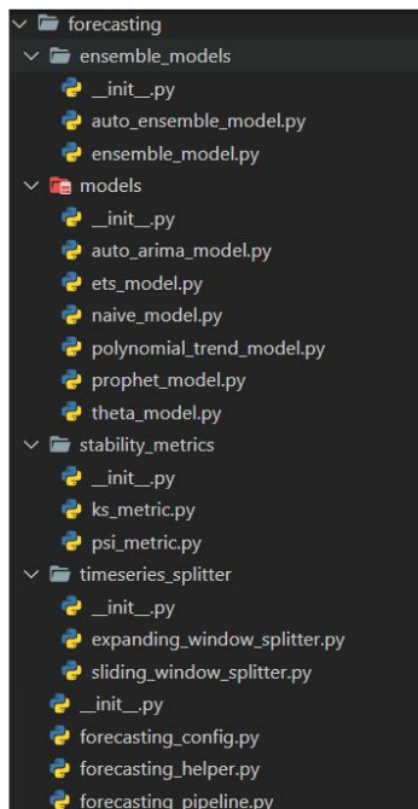
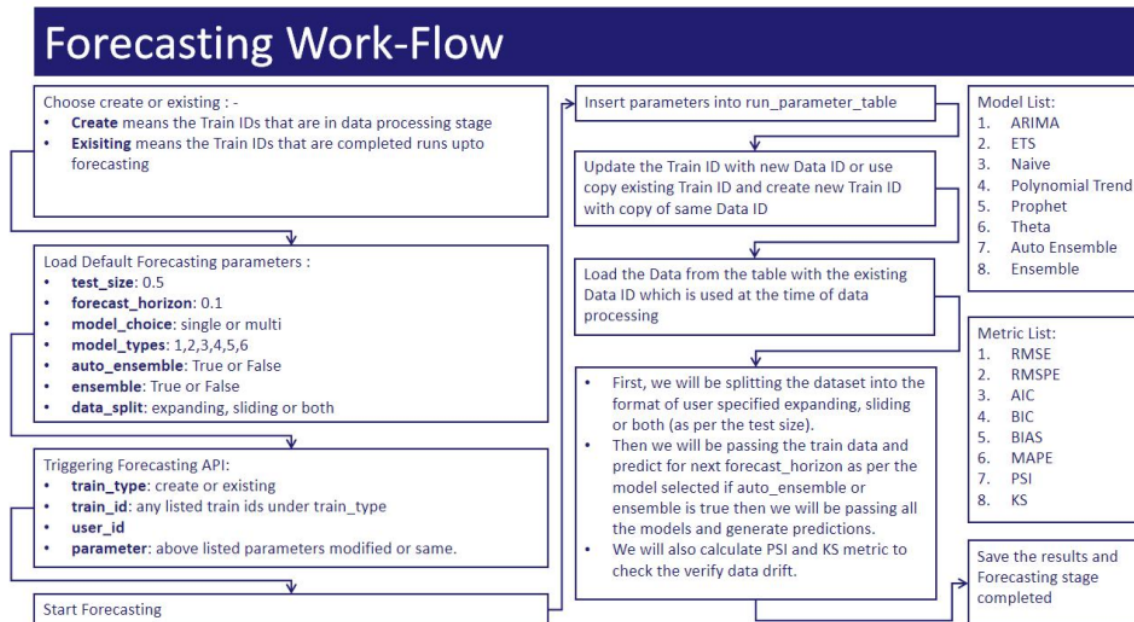
On left on side we can see the folder structure imputers, outliers are the folders that holds the imputation and outlier modules and to control them we are using data_processing_pipeline.py and to help with database control and configuration we have data_processing_config.py and data_processing_helper.py.

We have build this architecture to keep in mind the concept of scaling the solution and deploy it on a different environment easily as config and helper function are separate.

For further information on the imputation and outlier correction it is been discussed in the algorithms section of outlier correction and imputation.

3.5 Model Training Pipeline

Lets have a look on the forecasting workflow.



On the left hand side we can see folder structure for the forecasting module, here we can see ensemble_models, models, stability_metrics and timeseries_splitter. Again the architecture is very similar to data processing.

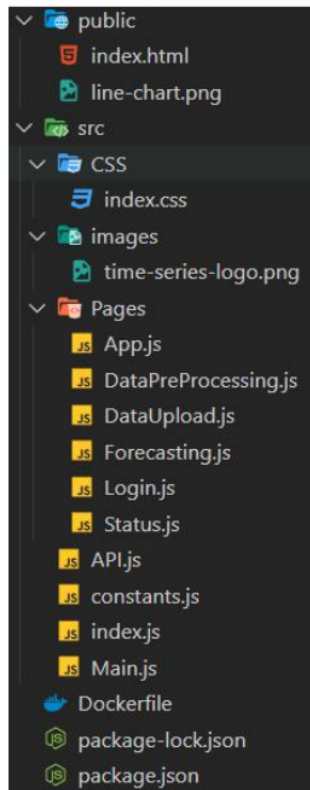
i. Models - here we have basic models as we can see here.

ii. Ensemble_models - there are two models one is auto_ensemble in which we pass basic models list and further we use gradient boosting regression for future predictions and in other model i.e. ensemble here also we pass basic models and after that we are using mean technique for aggregating the future predictions.

iii. Stability_metrics - here we are using concept drift metrics to check whether the train set and test set are within the limits of drift measure or not.

iv. Timeseries_splitter - here we have built expanding window splitter and sliding splitter, we will discuss more about this in detail later in the document.

3.6 - Web Front-end (using React) & Performance Visualization (using Stream-lit)



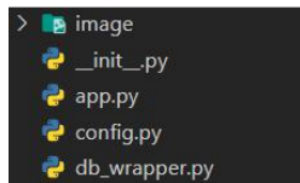
Here on left hand side you can see the folder structure for Web Front end written in reactjs.

i. Public - it holds the starting point of the we application

ii. Src - under source we have further division with css, images , Pages and couple of other files that holds the code for various pages like Login page, Data Upload Page, Data Pre-Processing page, Forecasting Page and Status Page.

iii. Constants.js and API.js - these files are helper files that are widely used in holding global variables and API function calls, which can used at multiple places to request for data from backend.

iv. Dockerfile - is used for deploying the application on separate container on either docker or kubernetes.



Here on left hand side you can see the folder structure for Stream-lit application written in python for visualization of the time series data, further metrics calculation and best model selection scoring. We all be seeing these screens in the demo section.

i. App.py - is the main entry point, where all the graphs and inferring are done.

ii. Config.py - stores the info same as the control, data processing and forecasting modules has

iii. Db_wrapper.py - its a database wrapper to fetch the data from the database.

4. Algorithms

As we are aware about the architecture of the solution, we would like to walk you through the models and techniques used behind the scenes to achieve this kind of solution.

4.1. Outlier Correction Techniques

We all know that we require an outlier detection module that clears the data points that are not expected to be in that point of time or if we say it's not a usual pattern.

6

Outliers can skew statistical measures and data distributions, providing a misleading representation of the underlying data and relationships. Removing outliers from training data prior to modeling can result in a better fit of the data.

Here what we will do is that we will detect those points and suppress it with NaN's and later we will impute it with following imputation techniques.

15

4.1.1. Local Outlier Factor

Local outlier factor (LOF) is an algorithm used for Unsupervised outlier detection. It produces an anomaly score that represents data points which are outliers in the data set. It does this by measuring the local density deviation of a given data point with respect to the data points near it.

18

4.1.2. Isolation Forest Outlier Correction

One efficient way of performing outlier detection in high-dimensional datasets is to use random forests. The Isolation Forest 'isolates' observations by randomly selecting a feature and then randomly selecting a split value between the maximum and minimum values of the selected feature.

4.1.3. Z-Score Outlier Correction

13

Z score is an important concept in statistics. Z score is also called standard score. This score helps to understand if a data value is greater or smaller than mean and how far away it is from the mean. More specifically, Z score tells how many standard deviations away a data point is from the mean.

4.2. Imputation Techniques

Once we have successfully detected the outliers, we need to fill them with either of the techniques chosen by the user mentioned among the lists mentioned below.

4.2.1. Mean

In mean imputation we will take mean of the whole time series and impute the NaN's with that value.

4.2.2. Median

In median imputation we will take median of the whole time series and impute the NaN's with that value.

4.2.3. Linear

In linear imputation we will linear interpolate the whole time series and impute the NaN's with those values.

4.2.4. Nearest

In nearest imputation we will find the nearest value either before Nan or after Nan and impute it with those values.

4.3. Time Series Models

After all the data processing techniques are done the data is been saved in the data table and is ready for forecast. Here are listed models that we have used for forecasting.

4.3.1. ARIMA

ARIMA is one the most commonly used statistical models for forecasting. Here we have used Auto ARIMA. The Auto ARIMA process seeks to identify the most optimal parameters for an ARIMA model, settling on a single fitted ARIMA model.

4.3.2. Exponential Smoothing

Exponential smoothing is a time series forecasting method for uni-variate data.

Time series methods like the Box-Jenkins ARIMA family of methods develop a model where the prediction is a weighted linear sum of recent past observations or lags.

Exponential smoothing forecasting methods are similar in that a prediction is a weighted sum of past observations, but the model explicitly uses an exponentially decreasing weight for past observations.

4.3.3. Prophet

Prophet is a procedure for forecasting time series data based on an additive model where non-linear trends are fit with yearly, weekly, and daily seasonality, plus holiday effects. It works best with time series that have strong seasonal effects and several seasons of historical data. Prophet is robust to missing data and shifts in the trend, and typically handles outliers well.

4.3.4. Naive

Naive forecasting is the technique in which the last period's sales are used for the next period's forecast without predictions or adjusting the factors. Forecasts produced using a naive approach are equal to the final observed value. The naive forecasting method is the easiest of all methods and it is suitable for finance and sales departments because it ensures that these departments work to improve the company.

4.3.5. Theta

Theta is another one of the most commonly used model for forecasting.

4.3.6. Poly-Trend

Forecast time series data with a polynomial trend. Default settings train a linear regression model with a 3rd degree polynomial transformation of the feature.

4.3.7. Ensemble

As we have seen the basic models and learned about there techniques. Some times the patterns are not captured with the base models in a correct manner, so here in ensemble we use base models as forecasting and we fuse the models result into one resulting in the best forecast. By default we are using mean aggregator but it can be changed to weighted mean or median.

4.3.8. Auto Ensemble

This is another ensemble technique where we use base models for forecasting and further we use another regression technique to forecast those results. Here it will optimize the weights for weighted mean.

4.3.9. Time Series Splitter

As we have walked through the base models and ensemble models, now we need to know how to split the data correctly so we can capture the best results with the above mentioned models.

10 4.3.9.1. Expanding Window

Split time series repeatedly into an growing training set and a fixed-size test set.

For example for window length = 5, step_length = 1 and forecast horizon = 3 here is a representation of the folds:

*	*	*	*	*	x	x	x	-	-	-				
*	*	*	*	*	*	x	x	x	-	-				
*	*	*	*	*	*	*	x	x	x	-				
*	*	*	*	*	*	*	*	x	x	x				

* = training fold.

x = test fold.

10 4.3.9.2. Sliding Window

Split time series repeatedly into a fixed-length training and test set.

For example for window length = 5, step_length = 1 and forecast horizon = 3 here is a representation of the folds:

*	*	*	*	*	x	x	x	-	-	-				
-	*	*	*	*	*	x	x	x	-	-				
-	-	*	*	*	*	*	x	x	x	-				
-	-	-	*	*	*	*	*	x	x	x				

* = training fold.

x = test fold.

4.4. Scoring Techniques

Now comes the most important part after the predictions are done, we need to calculate some metrics to detect the performance and concept drifts.

4.4.1. Concept Drift Metrics

17 Data can change over time. This can result in poor and degrading predictive performance in forecasting models that assume a static relationship between input and output variables.

This problem of the changing underlying relationships in the data is called concept drift in the field of machine learning.

4.4.1.1. Kolmogorov Smirnov (K-S)

3

This test compares the underlying continuous distributions $F(x)$ and $G(x)$ of two independent samples.

3

There are three options for the null and corresponding alternative hypothesis that can be selected using the alternative parameter.

two-sided: The null hypothesis is that the two distributions are identical, $F(x)=G(x)$ for all x ; the alternative is that they are not identical.

3

Note that the alternative hypotheses describe the CDFs of the underlying distributions, not the observed values. For example, suppose $x_1 \sim F$ and $x_2 \sim G$. If $F(x) > G(x)$ for all x , the values in x_1 tend to be less than those in x_2 .

If the KS statistic is small or the p-value is high, then we cannot reject the null hypothesis in favor of the alternative.

5

4.4.1.2. Population stability index (PSI)

It is a metric to measure how much a variable has shifted in distribution between two samples over time. It is widely used for monitoring changes in the characteristics of a population and for diagnosing possible problems in model performance — many a times, it's a good indication if the model has stopped predicting accurately due to significant changes in the population distribution.

5ULES

PSI < 0.1: No change. You can continue using the existing model.

PSI >=0.1: but less than 0.2 — Slight change is required.

PSI >=0.2: Significant change is required. Ideally, we should not use this model anymore.

4.4.2. Performance Metrics

We have seen the concept drift metrics, now we will be looking in some most commonly used performance metrics.

16

4.4.2.1. Mean Absolute Percentage Error (MAPE)

The mean absolute percentage error (MAPE) is a measure of how accurate a forecast system is. It measures this accuracy as a percentage, and can be calculated as the average absolute percent error for each time period minus actual values divided by actual values.

$$M = \frac{1}{n} \sum_{t=1}^n \left| \frac{A_t - F_t}{A_t} \right|$$

n is the number of fitted points,

A_t is the actual value,

F_t is the forecast value.

Σ is summation notation

4.4.2.2. Akaike Information Criterion (AIC)

2

Akaike's Information Criterion is usually calculated with software. The basic formula is defined as:

$$AIC = -2(\log\text{-likelihood}) + 2K$$

Where:

K is the number of model parameters (the number of variables in the model plus the intercept).

Log-likelihood is a measure of model fit. The higher the number, the better the fit. This is usually obtained from statistical output.

For small sample sizes ($n/K < \approx 40$), use the second-order AIC:

$$AIC = -2(\log\text{-likelihood}) + 2K + (2K(K+1)/(n-K-1))$$

Where:

n = sample size,

K = number of model parameters,

Log-likelihood is a measure of model fit.

2

4.4.2.3. Bayesian Information Criterion (BIC)

The Bayesian Information Criterion (BIC) is defined as

$$k \log(n) - 2\log(L(\theta)).$$

Here n is the sample size; the number of observations or number of data points you are working with. k is the number of parameters which your model estimates, and θ is the set of all parameters.

$L(\theta)$ represents the likelihood of the model tested, given your data, when evaluated at maximum likelihood values of θ . You could call this the likelihood of the model given everything aligned to their most favorable.

4.4.2.4. Bias

Bias-variance decomposition is extremely important if you want to get a really good grasp of things like over-fitting, under-fitting, and model capacity.

$$Bias(\hat{\theta}_S, \theta) = \mathbb{E}_{S \sim D^m} [\hat{\theta}_S] - \theta$$

Θ - true values

$E[\theta]$ - forecast values

4.4.2.5. Root Mean Squared Error (RMSE)

7

To compute RMSE, calculate the residual (difference between prediction and truth) for each data point, compute the norm of residual for each data point, compute the mean of residuals and take the square root of that mean. RMSE is commonly used in supervised learning applications, as RMSE uses and needs true measurements at each predicted data point.

$$RMSE = \sqrt{\frac{\sum_{i=1}^N \|y(i) - \hat{y}(i)\|^2}{N}},$$

7

where N is the number of data points, $y(i)$ is the i-th measurement, and $\hat{y}(i)$ is its corresponding prediction.

4.4.2.6. Root Mean Percentage Squared Error (RMSPE)

This is same as above the only difference is in error instead of mean error we take mean percentage error.

4.4.3. Best Model Selection Scoring Method

This is a custom model selection scoring method. Here we consider all the performance metrics.

Step 1: First we load all the metrics in column wise

Step 2: Then we create ranks of those metrics (absolute value) with respect to each time series id, split no and splitting window technique. Our ranks will be at model level. Lower the rank better is the model performance

Step 3: After the model ranks are created we generate weighted scores using the formulae of rank of that model / (sum(rank of all the models) + 1).

Step 4: We take an average of all the scores column wise.

Step 5: We sort the scores and find the model on each splits , time series id and splitting window.

Step 6: We find the most stable model by finding the lowest variance among the splits.

Step 7: We find the continuous improving model by checking if the second split is lesser than or equals to first split and so on.

Step 8: We find the overall lowest average model, here we average out all the scores and find the lowest score.

Note: We need to keep in mind that lower the score better the model performance.

5 - Deployment & Scalability

Now comes the most important part of solution and that is product-ionizing and scaling of the solution. Here we are using Docker to scale and deploy the solution. As we are in the last section we are mostly aware about the each components of the solution. All the above mentioned modules are scalable and flexible, as they are not tightly coupled and completely dockerized. They can simply be used as a single entity and scale up and down as per the volume of the time series.

Below we have added the docker deployment and scaling components along with screen shots.

```
db:
  build: ./mysql
  container_name: mysql_db
  command: --default-authentication-plugin=mysql_native_password
  environment:
    MYSQL_ROOT_PASSWORD: "root"
    MYSQL_DATABASE: "mlops_ts_fcst"
    MYSQL_USER: "mlops_user"
    MYSQL_PASSWORD: "MLOPS1234"
  ports:
    - "8306:3306"
  networks:
    - internal-network
  volumes:
    - dbdata:/var/lib/mysql

redis_db:
  image: redis:alpine3.15
  container_name: redis
  command: redis-server
  ports:
    - 6379:6379
  networks:
    - internal-network
  volumes:
    - redis_db_data:/data
```

```
celery_data_processing:
  build: ./python_image
  command: /home/conda/bin/celery -A data_processing.data_processing_pipeline worker --loglevel=INFO --autoscale=4,2
  image: python-image:1.0
  environment:
    - MKL_NUM_THREADS=1
    - PENBLAS_NUM_THREADS=1
    - BLIS_NUM_THREADS=1
    - OMP_NUM_THREADS=1
  depends_on:
    - redis_db
    - db
  deploy:
    replicas: 1
  networks:
    - internal-network
  volumes:
    - api_data:/app/
    - type: bind
    - source: ./backend/
    - target: /app/backend/
```

```
celery_forecasting:
  image: python-image:1.0
  command: /home/conda/bin/celery -A forecasting.forecasting_pipeline worker --loglevel=INFO --autoscale=4,2
  environment:
    - MKL_NUM_THREADS=1
    - PENBLAS_NUM_THREADS=1
    - BLIS_NUM_THREADS=1
    - OMP_NUM_THREADS=1
  depends_on:
    - redis_db
    - db
  networks:
    - internal-network
  deploy:
    replicas: 1
  volumes:
    - api_data:/app/
    - type: bind
    - source: ./backend/
    - target: /app/backend/
```

```
flower_web:
  image: python-image:1.0
  command: /home/conda/bin/celery --broker redis://redis_db:6379 --result-backend redis -A celery_app flower
  container_name: flower_monitor
  ports:
    - 5555:5555
  volumes:
    - api_data:/app/
    - type: bind
      source: ./backend/
      target: /app/backend/
  networks:
    - internal-network
  depends_on:
    - celery_forecasting
    - celery_data_processing
```

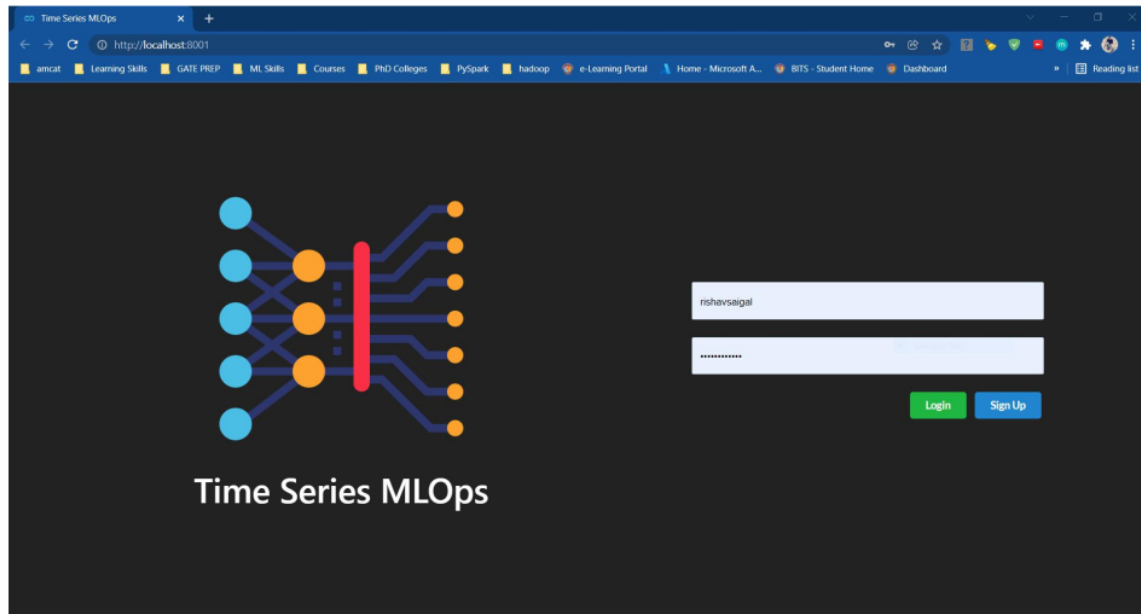
```
backend:
  image: python-image:1.0
  command: /home/conda/bin/uvicorn app:app --port 8000 --reload --workers 1 --host 0.0.0.0 --log-level debug
  container_name: api_container
  ports:
    - "8000:8000"
  depends_on:
    - redis_db
    - db
    - flower_web
    - celery_data_processing
    - celery_forecasting
  networks:
    - internal-network
  volumes:
    - api_data:/app/
    - type: bind
      source: ./backend/
      target: /app/backend/
```

```
frontend:
  build: ./frontend
  container_name: frontend_container
  command: serve -s build -p 8001
  image: node-image:1.0
  ports:
    - "8001:8001"
  depends_on:
    - visualization
  networks:
    - internal-network
  volumes:
    - frontend_data:/app/
    - type: bind
      source: ./frontend/
      target: /app/frontend/
  links:
    - backend
    - visualization
    - flower_web
```

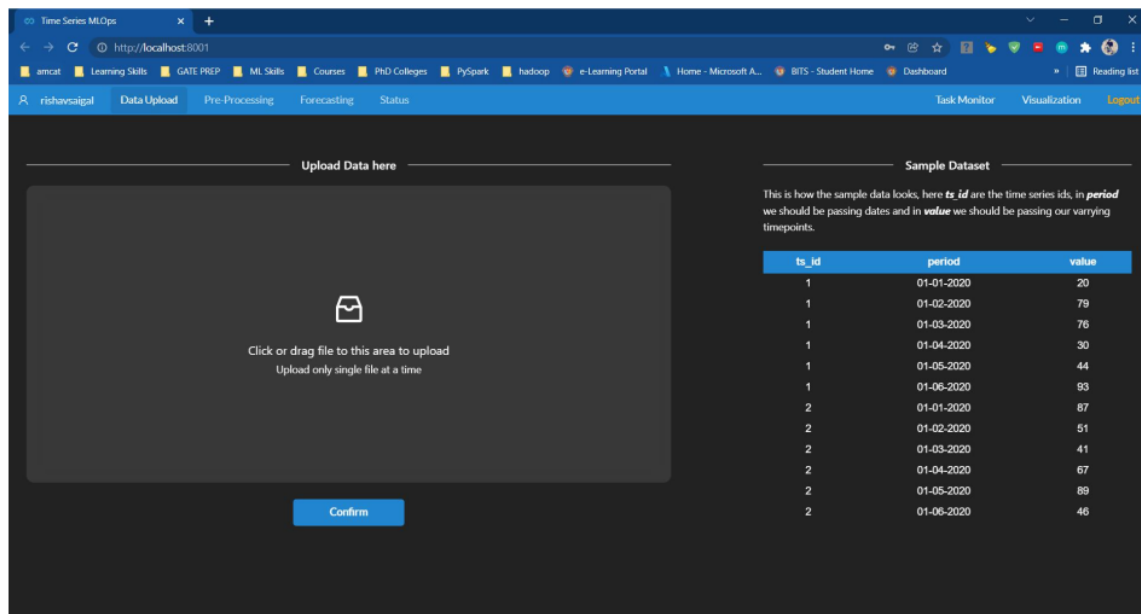
```
visualization:
  build: ./python_visual_image
  container_name: visual_container
  command: streamlit run app.py
  image: python-visual-image:1.0
  ports:
    - "8501:8501"
  depends_on:
    - backend
  networks:
    - internal-network
  volumes:
    - visualization_data:/app/
    - type: bind
      source: ./visualization/
      target: /app/visualization/
```


6 - Demo Screens

Login Page



Data Upload Page



Data Pre-Processing Page

The screenshot shows the 'Data Pre-Processing Parameters' page. The interface includes a top navigation bar with 'Data Upload', 'Pre-Processing', 'Forecasting', and 'Status' tabs. The 'Pre-Processing' tab is active. The page is divided into two main sections: 'Data Pre-Processing Parameters' and 'Sample Parameters'.

Data Pre-Processing Parameters:

- Choose type of Train:** A dropdown menu with 'create' selected.
- Select Train ID:** A dropdown menu with 'No Data' selected.
- Outlier Correction Choice:** A dropdown menu with 'if' selected.
- Contamination %:** A text input field with 'auto' entered.
- Imputation Choice:** A dropdown menu with 'linear' selected.
- Imputation If Zero:** A checkbox that is currently unchecked.
- Trigger Data Pre-Processing:** A blue button.

Sample Parameters:

- Outlier Choice Type:** A blue box containing the text: 'These are the list of outlier techniques currently available' followed by a bulleted list: 'if : Isolation Forest', 'lof : Local Outlier Factor', and 'zscore : Z-Score'.
- Contamination %:** A blue box containing the text: 'Either you can give "auto" or any percentage value but less than 1.'
- Z-Score Threshold:** A blue box containing the text: 'By default it is set to 3 but you can change it to any value.'
- Imputation Choice:** A blue box containing the text: 'These are the list of imputation techniques currently available.' followed by a bulleted list: 'linear', 'mean', 'median', and 'mode'.
- Imputation If Zero:** A blue box containing the text: 'By default it is false, but this is done to impute the zero values with imputation choice.'

Forecasting

The screenshot shows the 'Forecasting Parameters' page. The interface includes a top navigation bar with 'Data Upload', 'Pre-Processing', 'Forecasting', and 'Status' tabs. The 'Forecasting' tab is active. The page is divided into two main sections: 'Forecasting Parameters' and 'Sample Parameters'.

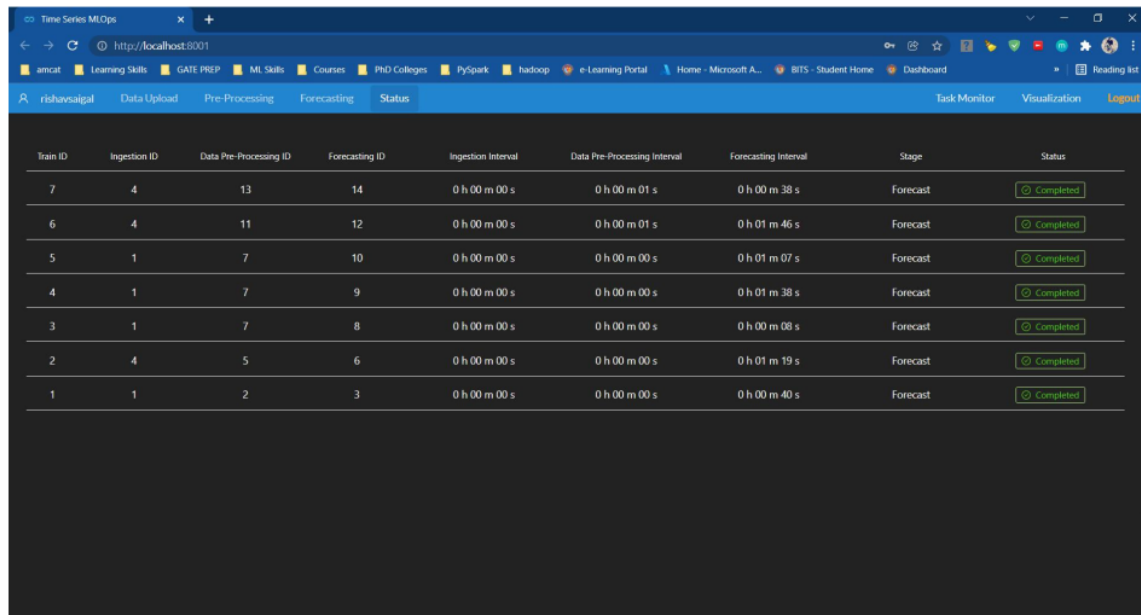
Forecasting Parameters:

- Choose type of Train:** A dropdown menu with 'create' selected.
- Select Train ID:** A dropdown menu with 'No Data' selected.
- Type of Training:** A dropdown menu with 'single' selected.
- Choose Model Type:** A dropdown menu with '1' selected.
- Type of Data Split:** A dropdown menu with 'expanding' selected.
- Test Size:** A text input field with '0.5' entered.
- Forecast Horizon:** A text input field with '0.1' entered.
- Auto Ensemble:** A checkbox that is currently unchecked.
- Ensemble:** A checkbox that is currently unchecked.
- Trigger Forecasting:** A blue button.
- Trigger Metric:** An orange button.

Sample Parameters:

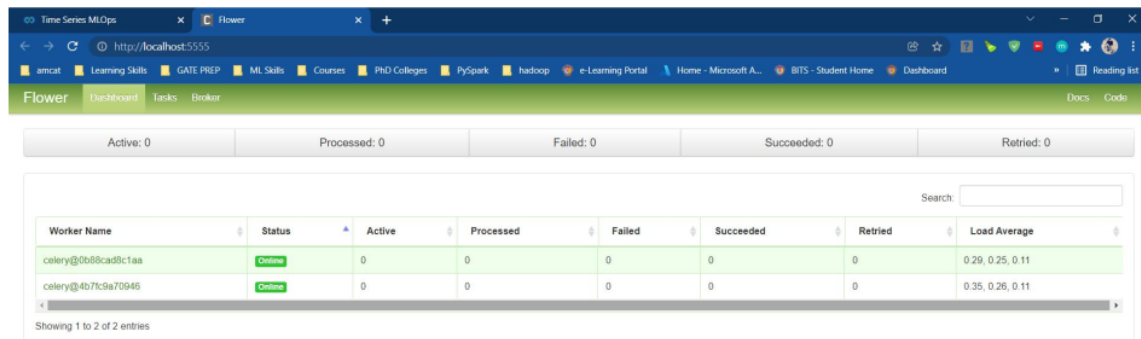
- Model Type:** A blue box containing the text: 'These are the list of models that currently supported by the system.' followed by a numbered list: '1. ARIMA', '2. Exponential Smoothing', '3. Naive', '4. Polynomial Trend', '5. Prophet', and '6. Theta'.
- Type of Data Split:** A blue box containing the text: 'Either you can split the data in expanding manner or sliding the window size or both.'
- Test Size:** A blue box containing the text: 'By Default the test size is 0.5 i.e 50% of the data will be used for testing.'
- Forecast Horizon:** A blue box containing the text: 'By default the forecast horizon is 0.1 i.e the forecast length will be 10% of the test data.'
- Auto-Ensemble or Ensemble:** A blue box containing the text: 'By default the auto-ensemble and ensemble are turned off. This is mostly used for enhancing the model results by aggregating the results or applying a tree based regression.'

Status



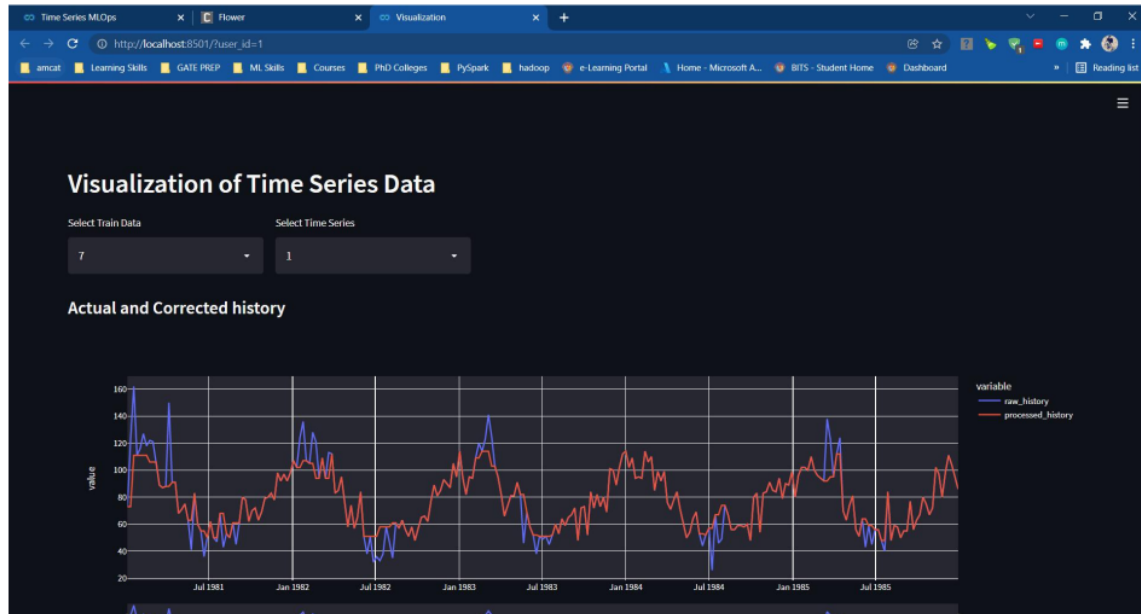
Train ID	Ingestion ID	Data Pre-Processing ID	Forecasting ID	Ingestion Interval	Data Pre-Processing Interval	Forecasting Interval	Stage	Status
7	4	13	14	0 h 00 m 00 s	0 h 00 m 01 s	0 h 00 m 38 s	Forecast	Completed
6	4	11	12	0 h 00 m 00 s	0 h 00 m 01 s	0 h 01 m 46 s	Forecast	Completed
5	1	7	10	0 h 00 m 00 s	0 h 00 m 00 s	0 h 01 m 07 s	Forecast	Completed
4	1	7	9	0 h 00 m 00 s	0 h 00 m 00 s	0 h 01 m 38 s	Forecast	Completed
3	1	7	8	0 h 00 m 00 s	0 h 00 m 00 s	0 h 00 m 08 s	Forecast	Completed
2	4	5	6	0 h 00 m 00 s	0 h 00 m 00 s	0 h 01 m 19 s	Forecast	Completed
1	1	2	3	0 h 00 m 00 s	0 h 00 m 00 s	0 h 00 m 40 s	Forecast	Completed

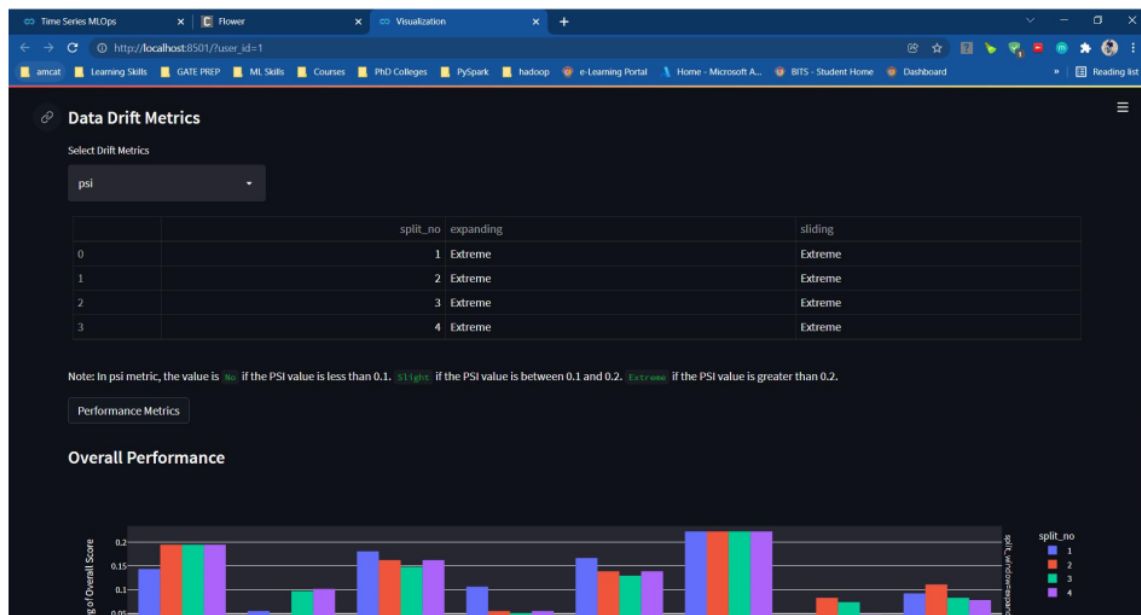
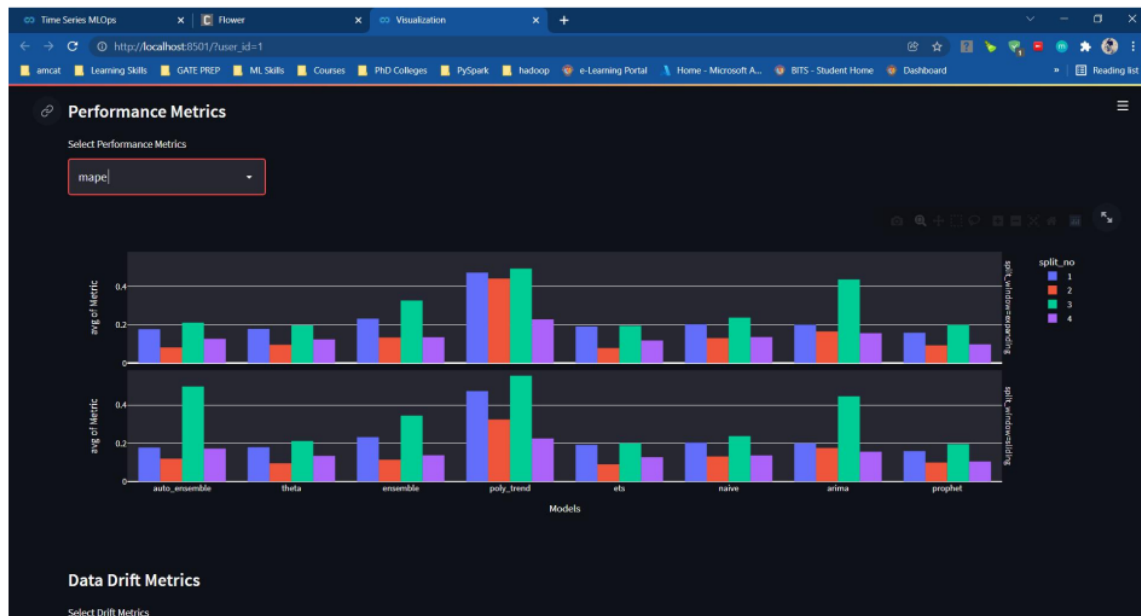
Task (Flower UI)

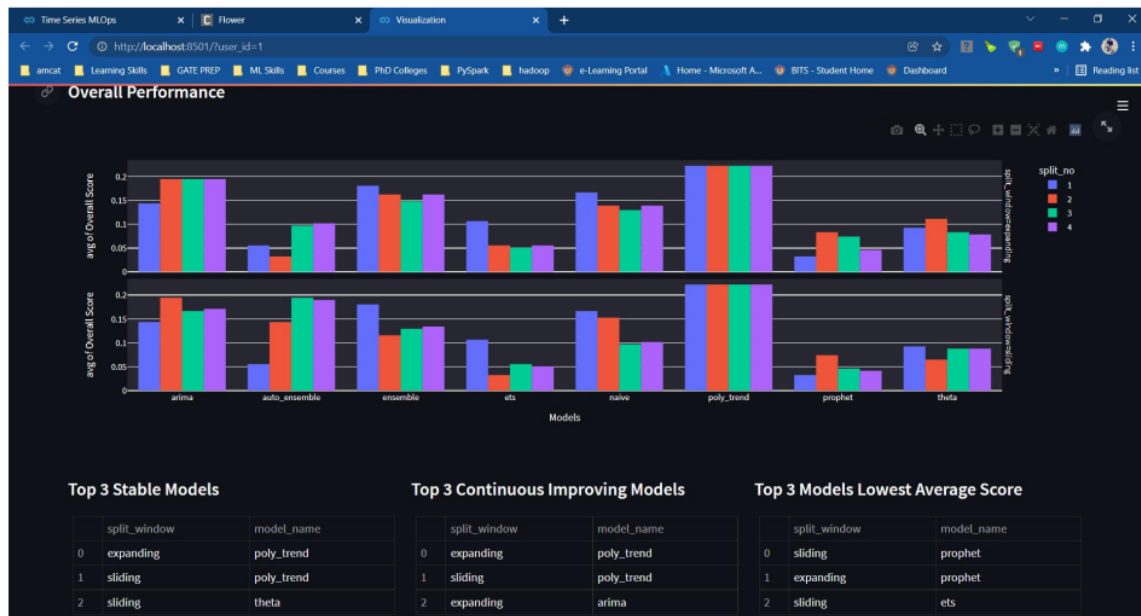


Worker Name	Status	Active	Processed	Failed	Succeeded	Retried	Load Average
celery@0b68cad3c1aa	Online	0	0	0	0	0	0.29, 0.25, 0.11
celery@4b7fc9a70946	Online	0	0	0	0	0	0.35, 0.26, 0.11

Visualization







Video Link

This link is only accessible within BITs Pilani University

<https://drive.google.com/file/d/15eC3Nr8x0DgNHZxEF45YQoiQ8g3WDDz8/view?usp=sharing>

7 - Conclusions and Recommendations

In this solution, we have successfully completed the problem statement for base version. But we have lot of areas of improvement like including more intelligent way of imputing the values, adding outlier correction, add more advance models like LSTM, RNN, GRU etc. Adding more insightful visualization at granular level.

8 - Reference

- Outlier Detector
 - <https://arundo-adtk.readthedocs-hosted.com/en/stable/notebooks/demo.html#OutlierDetector>
- Isolation Forest
 - <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- Local Outlier Factor
 - <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.LocalOutlierFactor.html>
- Isolation Forest
 - <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- Z-Score Outlier
 - <https://www.geeksforgeeks.org/z-score-for-outlier-detection-python/>
- Imputation
 - https://www.sktime.org/en/v0.8.0/api_reference/auto_generated/sktime.transformations.series.impute.Imputer.html
- Scikit Time for Model building and Time Series Split
 - https://www.sktime.org/en/v0.8.0/api_reference/forecasting.html
- Kolmogorov-Smirnov test
 - https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html
- Performance Metric
 - <https://www.statsmodels.org/stable/tools.html>
- Docker Creation
 - <https://docs.docker.com/get-started/overview/>
- Celery
 - <https://docs.celeryproject.org/en/stable/getting-started/index.html>
- Flower Web UI
 - <https://flower.readthedocs.io/en/latest/>
- React JS (Web)
 - <https://reactjs.org/docs/getting-started.html>
- Stream-lit (Python Web)
 - <https://docs.streamlit.io/>

9 - Duly Check List

a)	Is the Cover page in proper format?	Y / N
b)	Is the Title page in proper format?	Y / N
c)	Is the Certificate from the Supervisor in proper format? Has it been signed?	Y / N
d)	Is Abstract included in the Report? Is it properly written?	Y / N
e)	Does the Table of Contents page include chapter page numbers?	Y / N
f)	Does the Report contain a summary of the literature survey?	Y / N
i.	Are the Pages numbered properly?	Y / N
ii.	Are the Figures numbered properly?	Y / N
iii.	Are the Tables numbered properly?	Y / N
iv.	Are the Captions for the Figures and Tables proper?	Y / N
v.	Are the Appendices numbered?	Y / N
g)	Does the Report have Conclusion / Recommendations of the work?	Y / N
h)	Are References/Bibliography given in the Report?	Y / N
i)	Have the References been cited in the Report?	Y / N
j)	Is the citation of References / Bibliography in proper format?	Y / N

ORIGINALITY REPORT

27%

SIMILARITY INDEX

26%

INTERNET SOURCES

5%

PUBLICATIONS

18%

STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Birla Institute of Technology and Science Pilani Student Paper	3%
2	www.statisticshowto.com Internet Source	3%
3	docs.scipy.org Internet Source	2%
4	global.oup.com Internet Source	2%
5	towardsdatascience.com Internet Source	2%
6	medium.com Internet Source	2%
7	www.journal-aquaticscience.com Internet Source	1%
8	dokumen.pub Internet Source	1%
9	www.avercast.in Internet Source	1%

10	www.sktime.org Internet Source	1 %
11	worldwidescience.org Internet Source	1 %
12	hazelcast.com Internet Source	1 %
13	origin.geeksforgeeks.org Internet Source	1 %
14	web.archive.org Internet Source	1 %
15	www.ijirset.com Internet Source	1 %
16	findanyanswer.com Internet Source	1 %
17	machinelearningmastery.com Internet Source	1 %
18	docs.huihoo.com Internet Source	1 %
19	www.coursehero.com Internet Source	1 %
20	Submitted to Liverpool John Moores University Student Paper	1 %
21	www.infoworld.com Internet Source	

Exclude quotes	On	Exclude matches	< 1%
Exclude bibliography	On		